

Problem Log — Speckle DIC Development

Problem 1 — Images not found

What happened: [ERROR] No images found at: ./images\DSC_*.JPG

Cause: Windows uses backslashes \ in paths but Python treats \ as escape characters. Also the default folder path was wrong.

Solution: Use raw strings with r"" prefix and set the correct absolute path:

```
python
IMAGE_FOLDER = r"D:\Micro DIC Project\Speckle Pattern DIC\Images"
```

Problem 2 — Results folder not found

What happened: Script ran successfully but dic_results folder was nowhere to be found.

Cause: Python creates the folder relative to wherever it's *running from*, not where the script is saved. These are different locations in Jupyter/Spyder.

Solution: Use absolute path for results folder:

```
python
RESULTS_FOLDER = r"D:\Micro DIC Project\Speckle Pattern DIC\dic_results"
```

Problem 3 — Strain maps were extremely noisy

What happened: ϵ_{xx} maps showed random alternating red/blue patches instead of smooth fields.

Cause: Integer-pixel displacement resolution. When displacement jumps from 3px to 4px between adjacent grid points, the strain (gradient of displacement) shows artificial spikes. This is called the **peak-locking effect**.

Solution: Implemented **parabolic sub-pixel interpolation** — fit a parabola through the correlation peak and its neighbors to get fractional pixel accuracy (e.g. 3.73px instead of just 4px). This dramatically smoothed the strain maps.

Problem 4 — ROI too large (included machine housing)

What happened: The reference image showed the entire machine frame including black metal housing. Strain maps had artifacts from the grip regions.

Cause: ROI was set to crop the entire dog-bone sample including the wide grip shoulders and machine fixtures.

Solution: Crop only the **narrow gauge section** — the speckled white region between the two grips. Used Microsoft Paint cursor coordinates to find exact pixel boundaries: ROI = (973, 853, 1596, 982)

Problem 5 — ValueError: array too small for gradient

What happened:

ValueError: Shape of array too small to calculate a numerical gradient, at least (edge_order + 1) elements are required.

Cause: After cropping to narrow gauge ROI (129px tall), with SUBSET_SIZE=61 and MAX_SHIFT=50, the usable height was:

$$129 - 2 \times (30 + 50) = 129 - 160 = \text{negative!}$$

Zero grid rows remained in Y direction.

Solution: Reduced parameters to fit the thin ROI:

```
python
SUBSET_SIZE = 29 # half = 14px
MAX_SHIFT = 15 # total margin = 29px each side
# Usable height = 129 - 58 = 71px → ~14 grid rows ✓
```

Problem 6 — U displacement not monotonically increasing

What happened: Mean U displacement jumped randomly between frames:

Frame 001: U = -2.53px
Frame 002: U = +4.50px
Frame 007: U = +2.51px ← drops back
Frame 008: U = +6.39px ← jumps again

Cause: Camera/stage was vibrating between shots (rigid body motion). The entire specimen moved as a block, so mean displacement reflected camera shake, not deformation.

Solution v1: Subtract mean displacement per frame (rigid body correction). **But this over-corrected** — it removed actual elongation along with camera shake.

Solution v2 (correct): Use **grip-referenced displacement** — subtract only the displacement of the leftmost 3 grid columns (fixed grip side). This sets the left grip as zero and correctly measures elongation relative to it.

Problem 7 — Poisson's ratio was physically impossible ($\nu = 0.006$)

What happened: Estimated Poisson's ratio was nearly zero, meaning no transverse contraction was being detected.

Cause: The original wide ROI included grip regions where there is NO transverse strain. This swamped the actual ϵ_{yy} signal from the gauge.

Solution: Correct narrow gauge ROI + grip-referenced displacement. Once fixed, Poisson's ratio converged to $\nu = 0.406$ — physically realistic for metals.

Problem 8 — Max displacement hitting exactly MAX_SHIFT limit

What happened: Terminal output showed $\max_U = 30.00\text{px}$ and $\max_U = 50.00\text{px}$ — exactly the search limit for multiple consecutive frames.

Cause: The coarse NCC search was capped at MAX_SHIFT pixels. When real displacement exceeded this, the algorithm found the best match at the boundary, giving wrong results.

Solution: Progressively increased MAX_SHIFT:

```
python
# v1: MAX_SHIFT = 30 → capped at 30px
# v3: MAX_SHIFT = 15 → too small
# final: MAX_SHIFT = 25 → captures up to 50px with ICGN refinement
```

Problem 9 — Strain localization not visible in maps

What happened: ϵ_{xx} maps showed uniform green even at high strains (frame 28–30) where necking should produce a clear hot spot.

Cause: LSQ strain window of 7 grid points was too large, smoothing out the local strain concentration at the neck.

Solution: Reduce smoothing parameters:

```
python
STRAIN_WINDOW = 5 # from 7
STRAIN_SMOOTH = 0.8 # from 1.2
```

Problem 10 — ICGN refinement occasionally diverging

What happened: Some grid points showed displacement values far outside the expected range after ICGN refinement — the optimizer was diverging.

Cause: ICGN converges only if the initial guess (from coarse NCC) is close enough. At high strains, the speckle pattern distorts significantly and the coarse search sometimes picks a wrong local maximum.

Solution: Added sanity check — if ICGN result differs from coarse search by more than MAX_SHIFT, reject ICGN and keep coarse result:

```
python
if abs(dx - dx0) > max_shift or abs(dy - dy0) > max_shift:
    dx, dy, score = dx0, dy0, c0 # fall back to coarse
```

Combined with MAD-based outlier rejection to catch remaining bad points.